Mount Kenya University

Unlocking Infinite Possibilities

Student Name: Abdirahman Mohamud

Admission Number: bsccs/2024/54673

Course/Class: BSc Computer Science / Year 3

Unit Code: BIT4138

Unit Name: Advanced Cryptography

Lecturer Name: Micheal Nyoro

Semester/Academic Year: Semester 2

Project Title: Classical to Block Cipher Cryptographic Toolkit

Selected Technologies: Python, VS Code, Cryptographic Simulators, Statistical Tools

Online Portfolio / GitHub Link:

https://github.com/Abduz12/advanced/tree/main/advanced

Contents

Week 1: Cryptography Environment Setup

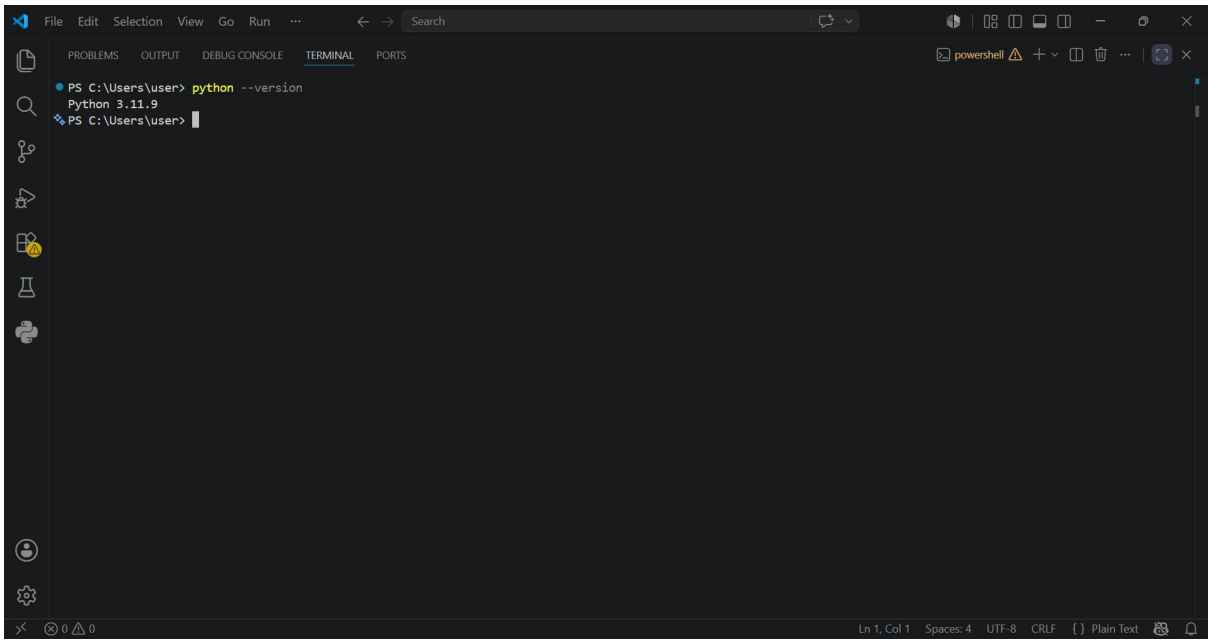

Fig 1: Installation of Python and VS Code

Fig 2: Installation of Cryptography Libraries

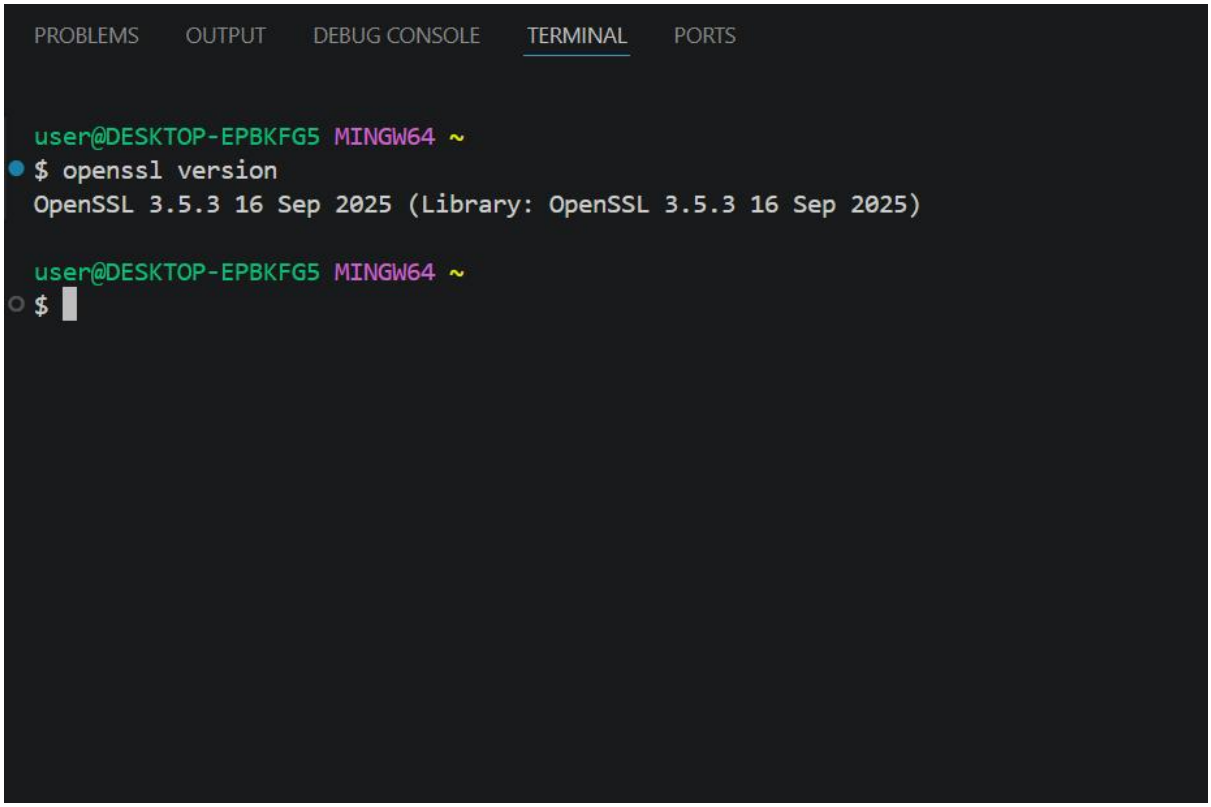

Fig 3: OpenSSL Installation Test

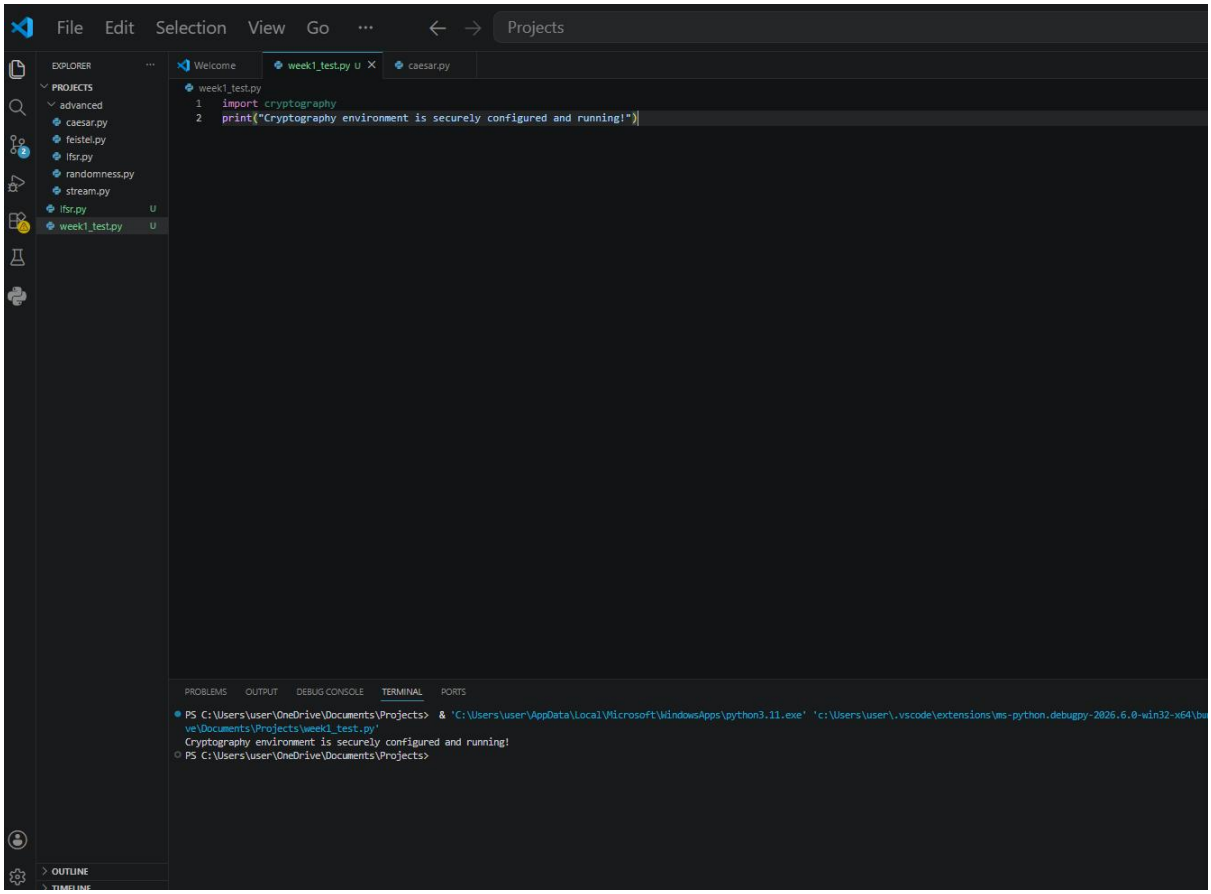

Fig 4: Local Encryption Script Execution

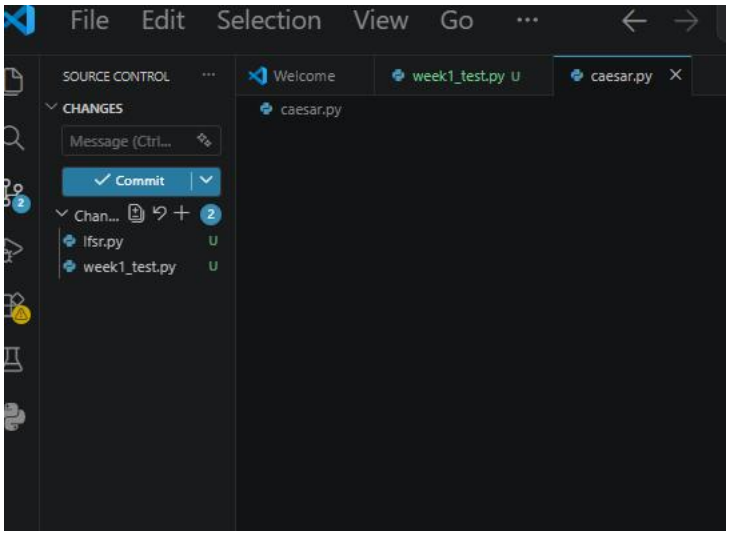

Fig 5: GitHub Repository Initialization

Student Reflection: Successfully installed and configured the cryptography development environment. Python, VS Code, and the Python Cryptography library were set up. OpenSSL was verified for secure key generation, and a GitHub repository was initialized to maintain professional version control and cybersecurity workflow reporting.

Week 2: Classical Cryptography and Cipher Design

Comparison and Implementation of Caesar and Vigenère Ciphers

Implemented Caesar and Vigenère ciphers in Python. I observed that the Vigenère cipher provides enhanced security over the Caesar cipher by utilizing multiple alphabets via a keyword, which effectively reduces redundancy and makes basic frequency analysis much more difficult for attackers.

Welcome caesar.py

caesar.py > ...

```
12  def vigenere_cipher(text, keyword, encrypt=True):
21          else:
22              result += char
23      return result
24
25  def brute_force_caesar(ciphertext):
26      print("--- Brute Forcing Caesar Cipher ---")
27      for shift in range(1, 26):
28          print(f"Shift {shift}: {caesar_cipher(ciphertext, shift, encrypt=False)}")
29
30  # Testing the Ciphers
31  plaintext = "SECURITY"
32  caesar_enc = caesar_cipher(plaintext, 4)
33  vig_enc = vigenere_cipher(plaintext, "KEY")
34
```

Fig 1: Python implementation of Caesar and Vigenère cipher functions.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
PS C:\Users\user\OneDrive\Documents\Projects>  & 'C:\Users\user\AppData\Local\Microsoft\WindowsApps\python3.11
.exe' 'c:\Users\user\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '6
3486' '--' 'c:\Users\user\OneDrive\Documents\Projects\def caesar_cipher(text, shift, encrypt=T.py'
Caesar (Shift 4) Encrypted: WIGYVMXC
Caesar Decrypted: SECURITY
Vigenere (Key 'KEY') Encrypted: CIAEVGDC
Vigenere Decrypted: SECURITY
--- Brute Forcing Caesar Cipher ---
Shift 1: VHFXULWB
Shift 2: UGEWTKVA
Shift 3: TFDVSJUZ
Shift 4: SECURITY
Shift 5: RDBTQHSX
Shift 6: QCASPGRW
```

Fig 2: Terminal output showing successful encryption and decryption of plaintext using both ciphers

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
PS C:\Users\user\OneDrive\Documents\Projects>  & 'C:\Users\user\AppData\Local\Microsoft\WindowsApps\python3.11
.exe' 'c:\Users\user\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '6
3486' '--' 'c:\Users\user\OneDrive\Documents\Projects\def caesar_cipher(text, shift, encrypt=T.py'
--- Brute Forcing Caesar Cipher ---
Shift 1: VHFXULWB
Shift 2: UGEWTKVA
Shift 3: TFDVSJUZ
Shift 4: SECURITY
Shift 5: RDBTQHSX
Shift 6: QCASPGRW
Shift 7: PBZROFQV
Shift 8: OAYQNEPU
Shift 9: NZXPMDOT
Shift 10: MYWOLCNS
```

Fig 3: Terminal output demonstrating a brute-force attack simulation on the Caesar Cipher.

Student Reflection:Implemented Caesar and Vigenère ciphers. I observed that the polyalphabetic Vigenère cipher provides enhanced security over the Caesar cipher by utilizing a keyword, effectively reducing redundancy and making basic frequency analysis much more difficult for attackers to execute.

Week 3: Stream Ciphers and Randomness Testing

Title: Implementation of Stream Cipher Systems and Random Sequence Analysis

Fig 1: Implementation of the Linear Congruential Generator (LCG) class in Python.

Fig 2: Terminal output displaying the pseudorandom sequence generated by the LCG.

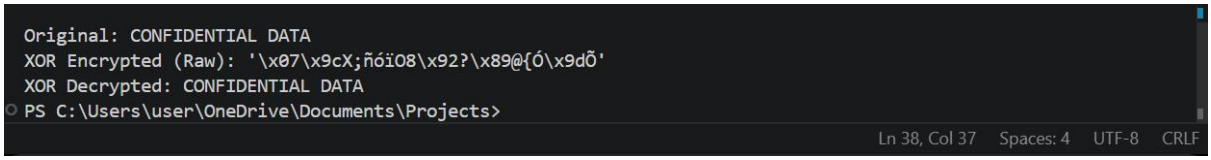

Fig 3: Terminal output showing the original, raw encrypted, and decrypted data using the XOR stream cipher.

Student Reflection:Developed a Linear Congruential Generator (LCG) and applied it to an XOR stream cipher. The exercises highlighted the critical difference between true randomness and pseudorandomness, emphasizing that stream cipher security relies heavily on the unpredictability and period of its keystream.

Week 4: Block Cipher Design and AES

Title: Advanced Encryption Standard (AES) and Block Cipher Operations

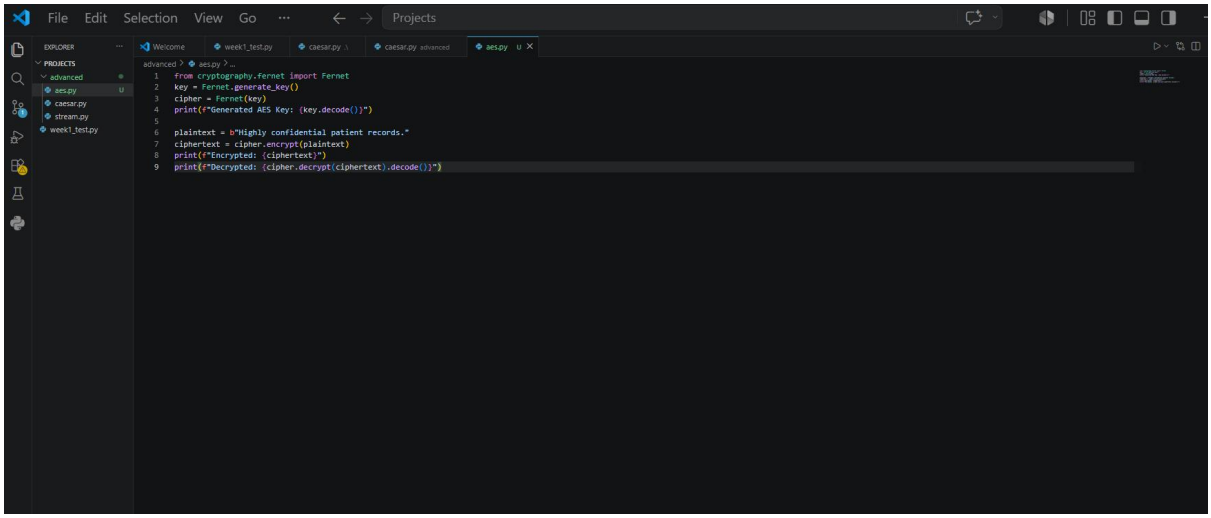

Fig 1: Python implementation of AES key generation and encryption using the Cryptography library.

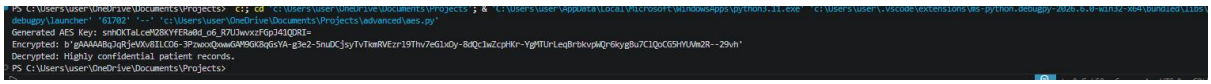

Fig 2: Terminal output showing the generated Symmetric Key and successful AES encryption/decryption.

Student Reflection: AES encryption was implemented to securely encrypt text and data using symmetric keys. The practical workflow demonstrated successful encryption, decryption, and secure key handling, emphasizing the necessity of robust key management in modern block cipher operations.

Week 5: Public Key Cryptography (RSA)

Title: Implementation of RSA Encryption and Key Management

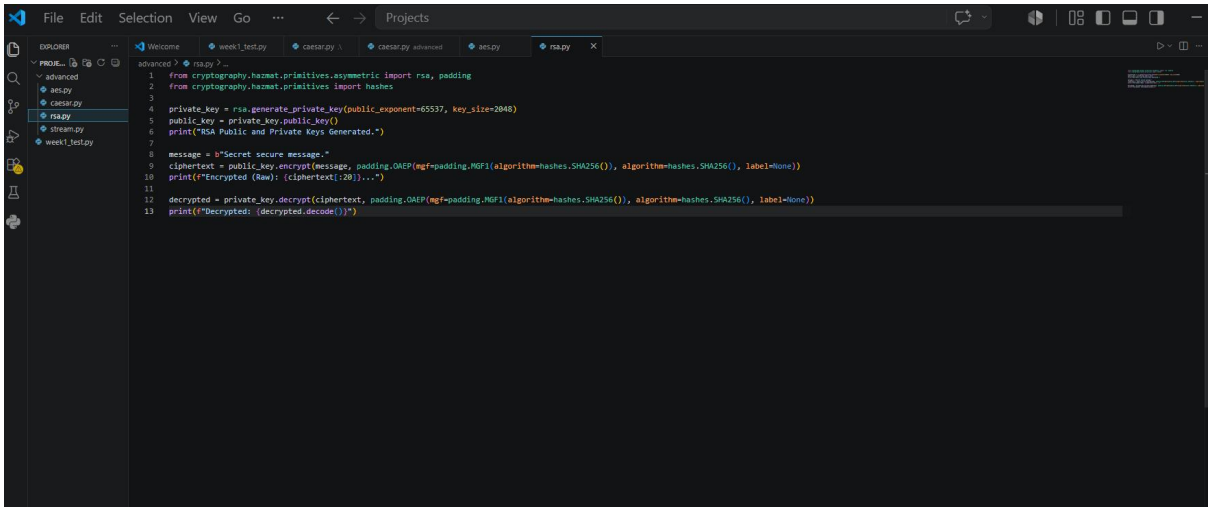

Fig 1: Python implementation of RSA public and private key pair generation.

Fig 2: Terminal output demonstrating successful encryption with the public key and decryption with the private key.

Student Reflection: RSA encryption demonstrated secure communication using asymmetric cryptography. The practical showcased the strict separation of public and private keys, ensuring that messages encrypted with a public key could only be successfully decrypted by the corresponding secure private key.